
Benchmarking Continual Agent Memory for Online Learning, Transfer, and Forgetting

Anonymous Author(s)

Affiliation

Address

email

Abstract

Effective long-term memory for LLM-based agents encompasses two fundamentally distinct capabilities: *procedural memory* (a.k.a. experiential memory), which enables agents to refine their reasoning and problem-solving skills over time, and *episodic memory* (a.k.a. factual memory), which allows agents to recall and retrieve past user interactions for personalization. In real-world deployments, these two memory types co-evolve with blurred boundaries—yet existing methods and benchmarks treat them in isolation, an assumption that breaks down the moment an agent must simultaneously execute tasks and serve a persistent user. We introduce **AgentMemoryBench**, the first benchmark to jointly evaluate procedural and episodic memory under a unified continual-learning framework. It spans four environment types (code-grounded, embodied, web-grounded, and dialogue-grounded) and standardizes five complementary evaluation modes (offline, online, replay, transfer, and repair) to measure improvement, retention, forgetting, generalization, and knowledge conflict resolution over time. Building on this benchmark, we propose **MEMs**, a multi-memory coordination methodology that maintains separate specialized stores and employs a lightweight trigger model as a meta-cognitive router to selectively retrieve and update each store. Experiments reveal that single-memory methods and in-context learning baselines suffer from systematic performance collapse in mixed-memory regimes due to memory contamination and architectural mismatch, while MEMs maintains stable learning across task boundaries, **outperforming the best single-memory baseline (EverMemOS) by 1.1–4.2 pp and surpassing mem0 and StreamICL by an average of ~15 pp**. AgentMemoryBench establishes a reproducible evaluation loop and provides practical guidance for developing memory systems that hold up in the real world. Source code: <https://anonymous.4open.science/r/AgentMemoryBench-CC4F>.

1 Introduction

Effective long-term memory for LLM-based agents encompasses two fundamentally distinct capabilities. *Procedural memory* (a.k.a. experiential memory) captures historical experiences and self-reflection insights, enabling the AI to refine its reasoning and problem-solving skills over time. *Episodic memory* (a.k.a. factual memory) enables memory retention beyond session limits, allowing the system to recall and retrieve past user interactions for personalization [Wu et al., 2025b, Hu et al., 2026c]. Both are prerequisites for continual adaptation: without procedural memory, agents repeat the same mistakes; without episodic memory, every session starts from scratch.

In real-world deployments, however, these two memory types co-evolve with blurred boundaries. Consider an agent that alternates between database query tasks and open-ended user dialogue: a procedural-memory method, e.g., AWM [Wang et al., 2024] can induce reusable SQL workflows from task trajectories but cannot track the user’s personal preferences; an episodic-memory method,

e.g., Mem0 [Chhikara et al., 2025] retains factual user context but cannot generalize procedural experience across task variants. Neither type alone is sufficient, as shown in Figure 1—yet existing methods and benchmarks treat them in isolation, an assumption that breaks down the moment an agent must simultaneously execute tasks and serve a persistent user [Zhi et al., 2025].

Single-memory methods work best when the required memory type matches what they store

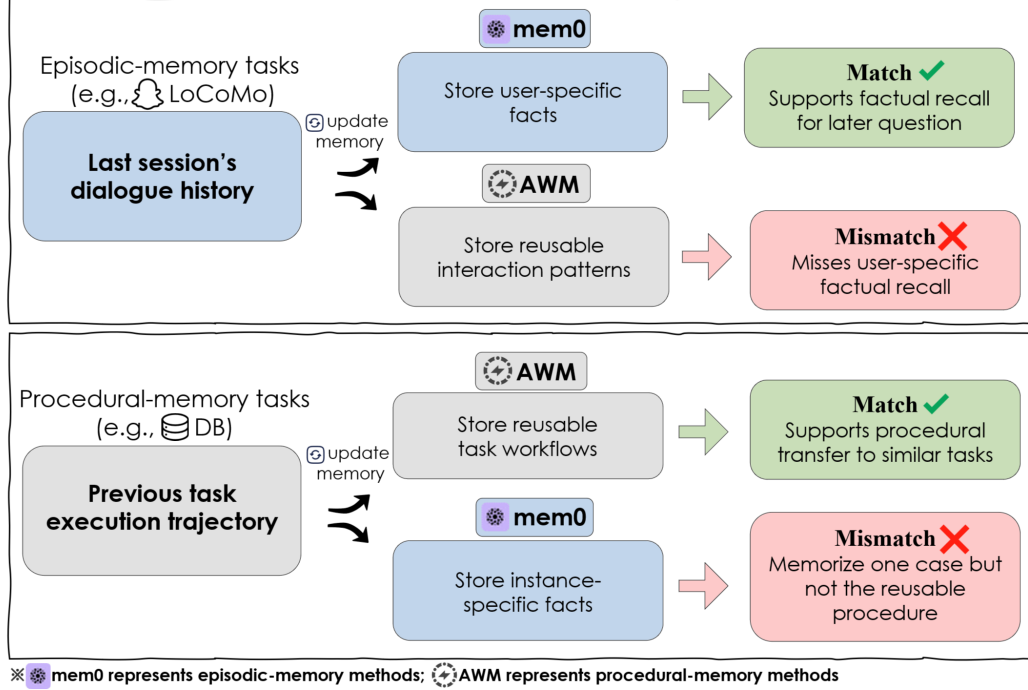


Figure 1: Procedural-memory and episodic-memory methods retain different forms of knowledge. On LoCoMo, mem0 [Chhikara et al., 2025] is well matched because success depends on retrieving user-specific facts. On DB, AWM [Wang et al., 2024] is well matched because success depends on reusing an abstract workflow. The mismatched method fails because it stores the wrong type of memory for the task demand.

Additionally, most existing memory benchmarks rely on static offline evaluation (e.g., LoCoMo [Maharana et al., 2024], MemoryBench [Ai et al., 2025], and MemoryAgentBench [Hu et al., 2026b]), which assesses performance only at a single time point. This setting is analogous to judging a learner solely by a one-time exam: while it can partially reflect how much knowledge has been acquired, it cannot reveal how that knowledge is accumulated, retained, forgotten, or adapted over time. As a result, offline evaluation obscures the learning dynamics of agent memory systems. Some more recent benchmarks adopt stream-based protocols (e.g., StreamBench [Wu et al., 2024], LifelongAgentBench [Zheng et al., 2025], and Evo-Memory [Wei et al., 2025]), moving closer to realistic continual-learning settings. However, they still do not provide a complete set of continual-learning metrics for systematically measuring key temporal properties such as knowledge retention, forgetting, and transfer [Shi et al., 2024]. However, existing benchmarks generally lack dedicated metrics for evaluating how agents resolve conflicts between newly acquired knowledge and previously stored knowledge. In particular, they do not include conflict-resolution measures analogous to the edit-success criteria introduced in KnowEdit [Zhang et al., 2024], which are essential for assessing whether a memory system can correctly update or repair outdated or contradictory knowledge.

Our contributions are threefold:

- We present **AgentMemoryBench**, the first unified benchmark that simultaneously evaluates procedural memory and episodic memory across five complementary modes (offline, online, replay, transfer, and repair, as illustrated in Figure 2) and four types of grounding environments in a single reproducible framework.

- We propose **MEMs**, a multi-memory coordination methodology that maintains separate specialized stores and employs a lightweight trigger model to route retrieval and updates, enabling stable procedural reuse and personalization under blurred task boundaries.
- Experiments reveal that single-memory methods and in-context learning baselines suffer from systematic performance collapse in mixed-memory regimes, as shown in the top part of Figure 2, due to memory contamination and architectural mismatch, while MEMs maintains stable learning across task boundaries, as shown in Figure 3.

2 Related Work

Memory methods vs. in-context learning. In-context learning (ICL) is a prompt-learning paradigm [Dou et al., 2026]: it incorporates a few input–output examples into the prompt, leveraging the pre-trained knowledge of large language models and allowing models to infer task format and expected behavior without updating any parameters. Memory methods operate on a fundamentally different principle: they extract knowledge from interaction trajectories with users and/or environments and persist it in external stores and follow a structured retrieve–form–evolve lifecycle (see Appendix D for a full lifecycle model), enabling agents to accumulate and reuse experience across tasks. Memory methods are further divided into *procedural memory* methods, which extract reusable procedural knowledge from task execution (e.g., MemRL [Zhang et al., 2026a] and AgeMem [Yu et al., 2026]), and *episodic memory* methods, which retain user-specific facts and preferences (e.g., Mem0 [Chhikara et al., 2025] and EverMemOS [Hu et al., 2026a]).

Benchmarks have driven rapid progress in agent memory, mirroring their role in advancing reasoning [Wu et al., 2025b], general task-solving [Liu et al., 2025], and agentic abilities [Yao et al., 2023, Shridhar et al., 2021]. However, existing agent memory benchmarks bifurcate into episodic memory (e.g., LoCoMo [Maharana et al., 2024] and MemoryAgentBench [Hu et al., 2026b]) and procedural memory (e.g., StreamBench [Wu et al., 2024]), each evaluated in isolation. This fragmentation still leaves a critical gap, as shown in Figure 1. Some methods that attempt to bridge both types—such as MemGen [Zhang et al., 2025], which unifies heterogeneous memory representations via latent token sequences—have no benchmark that supports the mixed-memory streams in which such coordination is actually needed. MemoryBench [Ai et al., 2025] partially addresses this by covering both memory types, but restricts evaluation to offline, static snapshots—concealing how memory quality evolves under continuous interaction.

Offline evaluation conceals learning dynamics. Most existing benchmarks adopt a static train-test split (e.g., LongMemEval [Wu et al., 2025a], KnowMe-Bench [Wu et al., 2026], PersonaMem-v2 [Jiang et al., 2025], DMR [Packer et al., 2024], and MemoryAgentBench [Hu et al., 2026b]), measuring only terminal performance. This snapshot cannot reveal how memory quality evolves over time. Stream-based protocols (e.g., StreamBench [Wu et al., 2024] and LifelongAgentBench [Zheng et al., 2025]) introduce online learning curves, but cumulative success rate alone is insufficient: ICL baselines such as StreamICL [Wu et al., 2024] can match dedicated memory methods on this metric by activating LLM pretraining priors rather than accumulating genuine knowledge. As shown in Tables 2 and 3, StreamICL can match or exceed dedicated memory methods under offline and basic online metrics. Thus, distinguishing persistent memory methods from StreamICL requires generalization metrics, continual learning indicators (forgetting, forward transfer), and knowledge-conflict resolution measures—none of which existing benchmarks provide.

Context-learning benchmarks offer a complementary perspective. CL-bench [Dou et al., 2026], CONTEXTBENCH [Li et al., 2026], and CODE2BENCH [Zhang et al., 2026b] study an analogous question within a single session: whether an LLM can acquire knowledge from context that is not already contained in its pretraining knowledge and immediately apply it to downstream tasks. Mem2ActBench [Shen et al., 2026] extends this setting to memory-grounded tool calls. However, because it only evaluates the *retrieve* stage of the memory retrieve–form–evolve lifecycle, without testing memory formation or evolution, it should not be regarded as a benchmark for persistent agent memory in the full sense. Even so, these benchmarks do not provide the continual-learning-style temporal probes needed to assess persistent memory over an agent’s lifetime. Our replay, transfer, and repair modes extend this evaluation philosophy to the long-term memory setting, providing the multi-dimensional protocol needed to rigorously assess persistent memory methods and distinguish them from in-context learning baselines. A detailed benchmark comparison is provided in Table 1.

Table 1: Comparison of AgentMemoryBench with existing benchmarks in terms of memory coverage, evaluation modes, and continual-learning capabilities. AgentMemoryBench is the only benchmark that jointly evaluates procedural and episodic memory while also supporting online, transfer, replay, and repair settings within a unified framework. Table definitions are provided in Appendix B.

Benchmark	Memory Eval	CL Metrics	Distinguish Learn	Hybrid Memory	Multi-turn	Tool-use	Single-task Online	Multi-task Online	Offline	Intra-env Transfer	Cross-env Transfer	Replay	Repair
AgentMemoryBench (ours)	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
LoCoMo [†] [Maharana et al., 2024]	✓	□	×	×	×	×	×	×	✓	×	×	×	×
HaluMem [‡] [Chen et al., 2026]	✓	×	×	×	×	×	✓	×	×	×	×	×	×
StreamBench [Wu et al., 2024]	✓	×	×	×	×	✓	✓	×	×	✓	×	×	×
LifelongAgentBench [Zheng et al., 2025]	✓	×	×	×	×	✓	✓	×	×	✓	×	×	×
Evo-Memory [Wei et al., 2025]	✓	×	×	×	✓	✓	✓	×	×	×	✓	×	×
MemoryBench [Ai et al., 2025]	✓	□	×	✓	✓	×	×	×	✓	×	×	×	×
MemoryAgentBench [Hu et al., 2026b]	✓	□	✓	×	×	×	×	×	✓	×	×	×	×
Mem2ActBench [Shen et al., 2026]	×	□	×	□	×	✓	□	□	□	□	□	□	□
CL-bench [§] [Dou et al., 2026]	×	□	✓	□	✓	✓	□	□	□	□	□	□	□

✓: supported; ×: not supported; □: not applicable

[†] Also includes LongMemEval [Wu et al., 2025a], KnowMe-Bench [Wu et al., 2026], PersonaMem-v2 [Jiang et al., 2025], and DMR [Packer et al., 2024]; [‡] Also includes AMemGym [Jiayang et al., 2026]; [§] Also includes CONTEXTBENCH [Li et al., 2026] and CODE2BENCH [Zhang et al., 2026b]

3 AgentMemoryBench

We address a fundamental evaluation gap—*can existing memory methods maintain stable performance when procedural and episodic tasks co-occur with blurred boundaries?*—by introducing **AgentMemoryBench**, which comprises six datasets (DB [Liu et al., 2025], OS [Liu et al., 2025], KG [Liu et al., 2025], ALFWorld [Shridhar et al., 2021], WebShop [Yao et al., 2023], and LoCoMo [Maharana et al., 2024]) across four environment types (code-grounded, embodied, web-grounded, and dialogue-grounded) and standardizes five complementary evaluation modes—offline, online, replay, transfer, and repair—to measure learning improvement, knowledge retention, forgetting, cross-task generalization, and conflict resolution. To complement the benchmark, we further propose **MEMs**, a multi-memory coordination methodology that employs a lightweight trigger model to route retrieval and updates across specialized system and personal memory stores.

Figure 2 illustrates the five evaluation modes supported by AgentMemoryBench—Offline, Online, Replay, Transfer, and Repair—each targeting a distinct capability axis of a memory system.

Offline Mode follows the standard train-test split (Figure 2, top). The agent first processes all training sub-tasks to update memory, then evaluates on the held-out test set. For procedural tasks (DB, OS, KG, ALFWorld, and WebShop), we report Average Success Rate (**ASR**), Average Steps on Success (**AS-succ**), and Average Steps on Failure (**AS-fail**); for episodic tasks (LoCoMo), we use **F1-score**, **BLEU-score**, and **LLM-as-Judge** score.

Online Mode streams sub-tasks to the agent one by one; the agent updates memory in real-time after each step. We record the Cumulative Success Rate (**CSR**) at every step to trace the full learning trajectory. Crucially, this mode supports both *single-memory* and *mixed-memory* scenarios—the latter, illustrated in the top row of Figure 2, exposes whether a memory system can handle heterogeneous tasks with blurred boundaries without interference.

Replay Mode partitions the sample stream into K stages. After each stage k , the agent is tested on a random subset of previously seen samples using memory and not update. The Forgetting Rate (**FR**) for a sample s is:

$$\text{FR}(s) = \max\left(0, \frac{P_{\text{imm}}(s) - P_{\text{replay}}(s)}{P_{\text{imm}}(s)}\right) \times 100\% \quad (1)$$

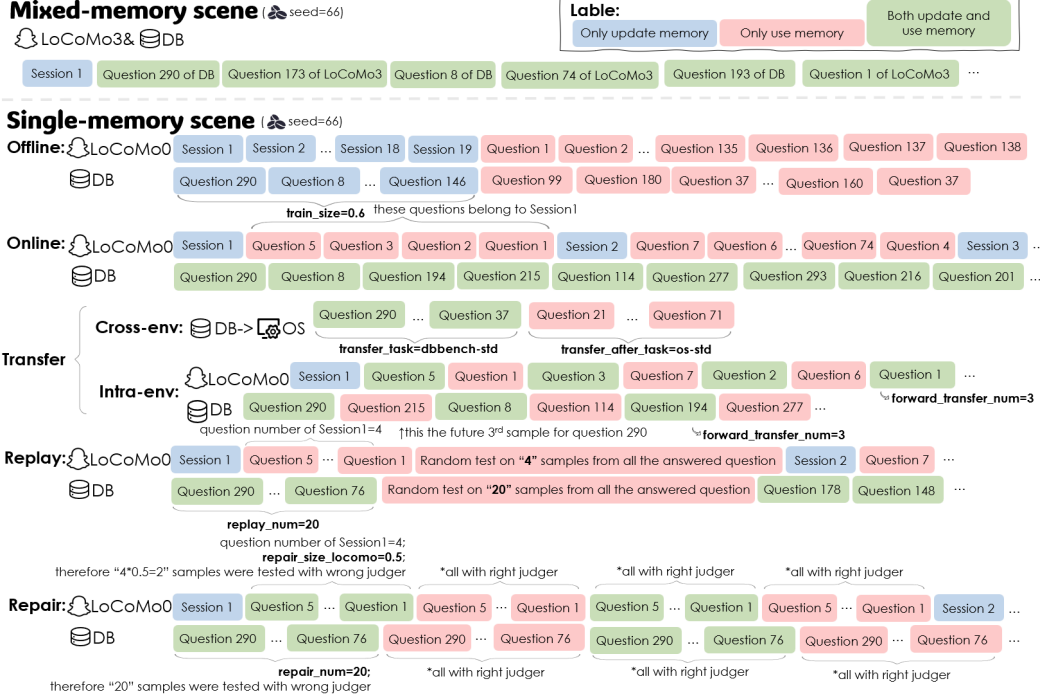


Figure 2: Overview of AgentMemoryBench evaluation pipelines. The top panel illustrates the mixed-memory scenario (LoCoMo&DB interleaved); the bottom panel details all five evaluation modes—Offline, Online, Transfer, Replay, and Repair—under the single-memory setting. **Boldface** text in the figure denotes experimental hyperparameters.

where $P_{\text{imm}}(s)$ is the score at first learning and $P_{\text{replay}}(s)$ is the score at a later replay stage. Overall FR is averaged across all samples and stages; $\text{FR} = 0$ means perfect retention, higher values signal catastrophic forgetting.

Transfer Mode covers two settings. *Cross-environment transfer* (TG) evaluates whether knowledge accumulated in one domain generalizes to another: the agent first runs online on a source task (e.g., DB), then is evaluated on a target task (e.g., OS) without further updates; TG is the performance difference against the OS-only baseline. *Intra-environment forward transfer* measures the predictive boost for future samples within the same stream after building memory from earlier ones:

$$\text{FTG} = \frac{1}{|\mathcal{P}|} \sum_{\substack{(i,j) \in \mathcal{P} \\ j > i}} (s(i \rightarrow j) - s(j)) \quad (2)$$

where $s(i \rightarrow j)$ is the score on sample j after building memory from sample i , and $\mathcal{P}$ is the set of all forward pairs. (Due to time constraints, cross-environment TG results are not presented in the table; we report only intra-environment FTG.) Note that the interpretation of FTG differs by task type. For *procedural* tasks, $\text{FTG} > 0$ is desirable, as accumulated memory directly benefits future steps. For *episodic* tasks (e.g., LoCoMo), however, correct answers require facts from the specific session—without first updating memory from that session’s content, the agent cannot answer correctly. Paradoxically, a more negative FTG here indicates stronger memory engagement—the method genuinely writes and retrieves memory, but without prior access to the target session’s content, resulting interference dominates. A near-zero FTG instead signals that the method bypasses memory entirely and relies on LLM pretraining priors.

Repair Mode evaluates robustness to erroneous feedback and self-correction capability. Each cycle has four phases visible in Figure 2: (1) learning under *wrong* rewards to contaminate memory; (2) testing with contaminated memory; (3) re-learning under *correct* rewards to repair; (4) testing post-repair. Repair Gain (**RG**) measures recovery:

$$\text{RG} = \bar{s}_{\text{right}} - \bar{s}_{\text{wrong}} \quad (3)$$

We report two variants: **RG-Std** is computed only over samples whose judger was directly flipped (standard contamination), while **RG-Full** covers the entire test set. A positive gap $\text{RG-Full} > \text{RG-Std}$ indicates a *radiation effect*—wrong knowledge has spilled over to non-contaminated samples, and successful repair also detoxifies them.

4 Experiments

4.1 Experiment settings

We compare against the following baselines. **Zero-shot** runs the agent with no memory. **Full-context** (episodic tasks only) feeds the entire conversation history directly as context. **StreamICL** [Wu et al., 2024] retrieves the top- k most similar past trajectories as in-context demonstrations via a streaming retrieval strategy. **mem0** [Chhikara et al., 2025] maintains a structured memory store with dynamic fact insertion, update, and deletion. **EverMemOS** [Hu et al., 2026a] adopts an OS-inspired architecture with self-organizing episode and profile stores. **AWM** [Wang et al., 2024] distills reusable workflow patterns from past trajectories into a structured workflow memory. **SkillNudge** (adapted from HermesAgent [Research, 2026]) extracts skill-level procedural knowledge from trajectories and injects it as soft nudges at inference time.

All memory methods use the best-practice hyperparameters reported in their respective papers: StreamICL (top_k=4, order=similar-at-top), mem0 (top_k=30, similarity_threshold=0), EverMemOS (top_k=10, search_method=hybrid, memory_types={episodic, profile}), AWM (top_k=8, order=similar-at-top), and SkillNudge (nudge_interval=5, max_selected_skills=3, max_manager_steps=10). For evaluation modes, offline mode uses a train/test split of 0.6/0.4; cross-environment transfer uses DB→OS and KG→ALFWorld as source/target pairs; intra-environment transfer sets forward_transfer_num=3; replay and repair both use a group size of 20, with repair_size_locomo=0.5 for LoCoMo tasks. Model backbone and random seed are reported in each table caption.

Table 2: **Performance comparison across modes (offline&online) and procedural tasks (DB, OS, KG, ALFWorld).** We report Average Success Rate (ASR), Average Steps for successful cases (AS-success), and Average Steps for failure cases (AS-failure). The best result in each column is marked in **bold** with **green background**. Performance changes relative to zero-shot baseline are shown in **blue** (improvement) or **red** (degradation). All memory methods adopt  GPT-4o-mini as the auxiliary model. For reproducibility, all experiments adopt a fixed random seed of 66.

Mode	Model	Memory	DB			OS			KG			ALFWorld		
			ASR↑	AS-succ↓	AS-fail↓	ASR↑	AS-succ↓	AS-fail↓	ASR↑	AS-succ↓	AS-fail↓	ASR↑	AS-succ↓	AS-fail↓
Offline 	Qwen3.5-72B	zero-shot	65.8	3.15	4.85	55.0	3.66	5.92	40.0	10.38	12.97	40.9	11.06	17.92
		streamICL	68.3(+2.5)	3.49(+0.34)	3.87(+0.98)	63.8(+8.8)	3.54(+0.12)	5.52(+0.40)	63.3(+23.3)	8.92(+1.46)	11.14(+1.83)	40.9(+0.0)	9.06(+2.00)	18.78(+0.86)
		mem0	68.3(+2.5)	2.85(+0.30)	3.37(+1.48)	56.9(+1.9)	3.12(+0.54)	5.08(+0.84)	45.0(+5.0)	9.22(+1.16)	12.58(+0.39)	52.3(+11.4)	9.48(+1.58)	18.81(+0.89)
		EverMemOS	70.8(+5.0)	3.65(+0.50)	4.57(+0.28)	58.6(+3.6)	3.53(+0.13)	5.04(+0.88)	61.7(+21.7)	8.68(+1.70)	11.35(+1.62)	77.3(+36.4)	9.47(+1.59)	18.20(+0.28)
		AWM	66.7(+0.9)	3.61(+0.46)	4.88(+0.03)	55.2(+0.2)	2.91(+0.75)	5.23(+0.69)	51.7(+11.7)	10.00(+0.38)	12.86(+0.11)	52.3(+11.4)	13.91(+2.85)	17.95(+0.03)
		SkillNudge	68.3(+2.5)	4.41(+1.26)	5.97(+1.12)	53.4(+1.6)	3.19(+0.47)	5.04(+0.88)	45.0(+5.0)	10.59(+0.21)	13.27(+0.30)	38.6(+2.3)	10.12(+0.94)	18.56(+0.64)
Online 	Qwen3.5-72B	zero-shot	68.7	3.23	4.86	51.4	3.47	5.59	42.7	9.97	13.09	42.2	9.61	18.38
		streamICL	73.0(+4.3)	3.96(+0.73)	5.01(+0.15)	57.6(+6.2)	3.27(+0.20)	4.69(+0.90)	62.0(+19.3)	8.64(+1.33)	10.54(+2.55)	73.4(+31.2)	7.84(+1.77)	16.61(+1.77)
		mem0	66.0(+2.7)	2.97(+0.26)	3.49(+1.37)	56.9(+5.5)	3.01(+0.46)	5.00(+0.59)	49.3(+6.6)	9.54(+0.43)	11.89(+1.20)	39.4(+2.8)	8.86(+0.75)	17.26(+1.12)
		EverMemOS	70.0(+1.3)	4.11(+0.88)	5.08(+0.22)	56.9(+5.5)	2.89(+0.58)	4.60(+0.99)	56.7(+14.0)	9.07(+0.90)	10.42(+2.67)	68.8(+26.6)	8.12(+1.49)	18.35(+0.03)
		AWM	68.3(+0.4)	3.85(+0.62)	5.72(+0.86)	52.1(+0.7)	3.17(+0.30)	5.13(+0.46)	50.0(+7.3)	9.95(+0.02)	12.04(+1.05)	40.4(+1.8)	10.36(+0.75)	18.94(+0.56)
		SkillNudge	70.3(+1.6)	4.17(+0.94)	5.16(+0.30)	53.5(+2.1)	3.12(+0.35)	5.12(+0.47)	51.3(+8.6)	9.61(+0.36)	12.78(+0.31)	44.0(+1.8)	10.10(+0.49)	17.89(+0.49)

4.2 Main Results

Tables 2 and 3 report performance across all baselines under offline and online modes. On procedural tasks, EverMemOS and mem0 consistently outperform other dedicated memory methods, with

Table 3: **Performance comparison on episodic tasks (LoCoMo0-3 only) across modes (offline & online). For LoCoMo tasks, we adopt the LoCoMo-refined standard [mem-eval-suite, 2026].** We report F1, BLEU, and LLM-as-Judge (LLM as J) scores across four question types. The best result in each column is marked in **bold** with green background. Performance changes relative to full-context baseline are shown in blue (improvement) or red (degradation). All memory methods adopt GPT-4o-mini as the auxiliary model, and Qwen3-14B is utilized as the judge model for all LoCoMo tasks. For reproducibility, all experiments adopt a fixed random seed of 66.

Mode	Model	Memory	Single-hop			Temporal			Multi-hop			Open-domain		
			F1↑	BLEU↑	LLM as J↑	F1↑	BLEU↑	LLM as J↑	F1↑	BLEU↑	LLM as J↑	F1↑	BLEU↑	LLM as J↑
Offline	Qwen3.5-72B	full-context	53.17	44.14	70.37	52.23	46.59	54.4	45.68	38.29	77.78	59.82	49.99	85.33
		streamICL	26.91 _(-26.26)	19.33 _(-24.81)	27.16 _(-43.21)	41.3 _(-10.93)	35.9 _(-10.69)	48.8 _(-5.6)	23.17 _(-22.51)	16.99 _(-21.3)	38.89 _(-38.99)	40.62 _(-19.2)	34.33 _(-15.66)	57 _(-28.33)
		mem0	34.39 _(-18.78)	27.02 _(-17.12)	48.15 _(-22.22)	13.82 _(-38.41)	9.8 _(-36.79)	12 _(-42.4)	17.06 _(-28.62)	13.4 _(-24.89)	33.33 _(-44.45)	32.01 _(-27.81)	27.27 _(-22.72)	42.33 ₍₋₄₃₎
		EverMemOS	38.89 _(-14.26)	30.29 _(-13.85)	55.56 _(-44.81)	17.28 _(-34.95)	14.92 _(-31.67)	4 _(-50.4)	24.77 _(-20.91)	18.44 _(-19.85)	50 _(-27.78)	44.59 _(-15.23)	37.28 _(-12.71)	65.33 ₍₋₂₆₎
		AWM	1.95 _(-31.22)	1.39 _(-42.75)	0 _(-70.37)	1.61 _(-50.62)	1.37 _(-45.22)	0 _(-54.4)	5.02 _(-40.66)	3.59 _(-34.7)	5.56 _(-72.22)	1.86 _(-57.96)	1.22 _(-48.77)	0 _(-85.33)
		SkillNudge	0.93 _(-52.24)	0.72 _(-43.42)	0 _(-70.37)	1.76 _(-50.47)	1.49 _(-45.1)	0 _(-54.4)	2.94 _(-42.74)	2.19 _(-36.1)	5.56 _(-72.22)	1.3 _(-58.52)	0.79 _(-49.2)	0 _(-85.33)
Online	Qwen3.5-72B	full-context	50.57	43.04	69.14	51.47	43.84	71.2	43.8	37.68	77.78	62.74	53.23	87
		streamICL	31.23 _(-19.34)	23.83 _(-19.21)	28.4 _(-40.74)	45.25 _(-6.22)	39.71 _(-4.13)	64 _(-7.2)	31.99 _(-11.81)	27.75 _(-9.93)	44.44 _(-33.34)	49.11 _(-13.63)	41.05 _(-12.18)	68.33 _(-18.67)
		mem0	31.44 _(-19.13)	24.07 _(-18.97)	33.33 _(-35.81)	10.84 _(-40.63)	6.72 _(-37.12)	24.8 _(-46.4)	19.41 _(-24.39)	15.78 _(-21.9)	22.22 _(-55.56)	23.24 _(-39.5)	20.02 _(-33.21)	33 ₍₋₅₄₎
		EverMemOS	40.36 _(-10.21)	32.49 _(-10.55)	56.79 _(-42.35)	15.35 _(-36.12)	13.15 _(-30.69)	4.8 _(-66.4)	22.46 _(-21.34)	18.13 _(-19.55)	44.44 _(-33.34)	43.47 _(-19.27)	36.26 _(-16.97)	59.67 _(-27.33)
		AWM	1.95 _(-48.62)	1.39 _(-41.65)	0 _(-69.14)	1.56 _(-49.91)	1.33 _(-42.51)	0 _(-71.2)	5.72 _(-38.08)	4.04 _(-33.64)	5.56 _(-72.22)	1.72 _(-61.02)	1.12 _(-52.11)	0 ₍₋₈₇₎
		SkillNudge	0.93 _(-49.64)	0.72 _(-42.32)	0 _(-69.14)	1.76 _(-49.71)	1.49 _(-42.35)	0 _(-71.2)	2.94 _(-40.86)	2.19 _(-35.49)	5.56 _(-72.22)	1.3 _(-61.44)	0.79 _(-52.44)	0 ₍₋₈₇₎

191 EverMemOS reaching 70.0% ASR on DB and 68.8% on ALFWorld in online mode. On episodic
192 tasks, full-context sets an upper bound that no memory method approaches across all four question
193 types, while AWM and SkillNudge nearly collapse to zero. These results surface four questions
194 we analyze in turn: **(Q1)** why do mem0 and EverMemOS—designed for personal conversation
195 memory—excel at procedural tasks? **(Q2)** AWM and SkillNudge both extract reusable knowledge
196 from trajectories—but what drives their performance gap? **(Q3)** Why does full-context dominate
197 every memory method on LoCoMo across all metrics? **(Q4)** StreamICL stores raw trajectory snippets
198 without abstraction—why does it match or exceed EverMemOS and mem0 on several procedural
199 metrics?

200 4.3 Analysis

201 **(Q1) Episode memory is the operative module for procedural tasks.** Inspecting the EverMemOS
202 codebase reveals that it models user memory as two components: *profile* (stable user attributes) and
203 *episode* (short-term interaction records). Running EverMemOS in online mode on OS and KG (294
204 samples, ~1,900 agent–environment interaction steps) yields **0** profile segments but **200+** episode
205 entries—procedural tasks produce no stable user-level attributes, so all memory writes land in the
206 episode store. The ablation in Table 4 confirms this: removing the episode store causes ASR to
207 drop by 1.3–14.7 pp across tasks, with the sharpest degradation on KG (−12.7 pp) and ALFWorld
208 (−14.7 pp). mem0, while architecturally different, maintains a structurally similar episode-like store,
209 explaining its comparable advantage over methods that lack this mechanism.

210 **(Q2) Skill routing beats workflow retrieval; episode augmentation amplifies the gain.** To test
211 whether AWM’s underperformance stems from retrieval noise at topk=8, we ablate it to topk=3; ASR
212 shifts by at most 0.7 pp, ruling this out as the root cause (Table 4). The gap is instead architectural:
213 AWM injects the top-*k* workflows verbatim by query similarity, risking context pollution with
214 redundant patterns. SkillNudge instead lets the agent *self-select* which skills to invoke and discloses
215 them progressively, preserving context quality while granting the agent more autonomy. Motivated
216 by Q1, we further augment SkillNudge with streamICL (topk=2) as a lightweight episode buffer.
217 The gains are striking: ASR rises by 4.7, 7.6, 12.7, and 31.2 pp on DB, OS, KG, and ALFWorld
218 respectively—uniformly surpassing standalone streamICL (topk=4) while consuming fewer in-context
219 tokens.

Table 4: **Ablation study on procedural memory tasks.** Each memory method is paired with its ablation variant (indented). Performance changes relative to each method’s main variant are shown in **blue** (improvement) or **red** (degradation). All memory methods adopt  Qwen3.5-27B as the main model, and  GPT-4o-mini as the auxiliary model. For reproducibility, all experiments adopt a fixed random seed of 66.

Memory	DB			OS			KG			ALFWorld		
	ASR↑	AS-succ↓	AS-fail↓	ASR↑	AS-succ↓	AS-fail↓	ASR↑	AS-succ↓	AS-fail↓	ASR↑	AS-succ↓	AS-fail↓
zero-shot	68.7	3.23	4.86	51.4	3.47	5.59	42.7	9.97	13.09	42.2	9.61	18.38
streamICL (topk=4)	73	3.96	5.01	57.6	3.27	4.69	62	8.64	10.54	73.4	7.84	16.61
AWM (topk=8)	68.3	3.85	5.72	52.1	3.17	5.13	50	9.95	12.04	40.4	10.36	18.94
- w. topk=3	68.0 _(-0.3)	3.67 _(-0.18)	4.82 _(-0.90)	51.4 _(-0.7)	2.78 _(-0.39)	5.24 _(+0.11)	49.3 _(-0.7)	10.07 _(+0.12)	12.18 _(+0.14)	40.4 ₍₀₎	10.25 _(-0.11)	18.45 _(-0.49)
EverMemOS (episode+profile)	70	4.11	5.08	56.9	2.89	4.6	56.7	9.07	10.42	68.8	8.12	18.35
- w/o. episode	68.7 _(-1.3)	3.87 _(-0.24)	5.06 _(-0.02)	52.8 _(-4.1)	3.45 _(+0.56)	5.26 _(+0.66)	44 _(-12.7)	10.03 _(+0.96)	12.65 _(+2.23)	54.1 _(+14.7)	8.83 _(+0.71)	18.26 _(-0.09)
SkillNudge	70.3	4.17	5.16	53.5	3.12	5.12	51.3	9.61	12.78	44	10.1	17.89
- w. streamICL (topk=2)	75 _(+4.7)	4.08 _(-0.09)	4.07 _(-1.09)	61.1 _(+7.6)	3.3 _(+0.18)	5.2 _(+0.08)	64 _(+12.7)	8.4 _(-1.21)	10.50 _(-2.28)	75.2 _(+31.2)	8.5 _(-1.6)	18.11 _(+0.22)

(Q3) **Full-context exploits a structural limitation of LoCoMo.** We measure the average memory length consumed by full-context on LoCoMo: $\sim 13,000$ tokens in offline mode and $\sim 24,000$ tokens in online mode. Both are well within the 256K context window of Qwen3.5-27B, meaning the model attends over the complete conversation history without any compression loss. The dominance of full-context is therefore not a failure of memory methods but a dataset artifact: LoCoMo dialogues are short enough that brute-force context stuffing remains the optimal strategy, making the dataset an inadequate testbed for evaluating memory under realistic length pressure. Nevertheless, this finding itself is informative: it establishes a clear diagnostic criterion—a benchmark must include dialogues that exceed the model’s effective attention horizon before memory compression becomes necessary.

(Q4) **Surface metrics conceal memory quality: the case for continual learning evaluation.** StreamICL leads procedural ASR (e.g., 73.0% on DB online), yet Table 5 reveals it is not a genuine memory system. For episodic tasks, FTG interpretation is reversed (see Section 3): near-zero FTG signals guessing via LLM priors, not real memory transfer. StreamICL’s FTG (-8.96 , closest to zero) confirms this—it also suffers the highest forgetting ($FR=29.41$) and the only negative repair gain ($RG\text{-Std}=-3.17$). EverMemOS’s more negative FTG (-14.18) reflects genuine memory engagement—interference from sessions whose content was never seen dominates, not memory failure; it leads on repair ($RG\text{-Std}=3.17$, $RG\text{-Full}=4.38$) and mem0 achieves the lowest forgetting ($FR=8.33$). CL metrics thus expose what surface accuracy conceals.

Table 5: **Continual learning metrics for episodic memory tasks (LoCoMo0).** The best performance is in **boldface**, and the second best is underlined. All memory methods adopt  Qwen3.5-27B as the main model, and  GPT-4o-mini as the auxiliary model. For reproducibility, all experiments adopt a fixed random seed of 66.

Memory	FR↓	FTG↓	RG-Std↑	RG-Full↑
streamICL	29.41	-8.96	-3.17	-0.73
mem0	8.33	<u>-9.70</u>	<u>1.59</u>	<u>0.00</u>
EverMemOS	<u>19.23</u>	-14.18	3.17	4.38

4.4 Mixed-Memory Evaluation

The single-task evaluations above reveal that no existing method handles both episodic and procedural memory well simultaneously: methods with strong episodic memory (mem0, EverMemOS) underperform on procedural tasks relative to their potential, while procedural-focused methods (AWM, SkillNudge) collapse entirely on LoCoMo. We therefore introduce a *mixed-memory* evaluation setting in which episodic tasks (LoCoMo) and procedural tasks (OS, ALFWorld, KG, DB) are interleaved

in a single stream, requiring the agent to handle both memory types concurrently without knowing which type each incoming task belongs to.

To address this, we propose **MEMs** (Multi-memory Coordination), which employs a lightweight trigger model to classify each incoming task and route it to the appropriate memory store—either an episodic memory module or a procedural memory module—for both retrieval and update. At inference time, MEMs queries both stores and fuses the retrieved context before passing it to the agent, allowing it to draw on whichever memory type is relevant without manual task-type annotation.

As shown in Figure 3, MEMs achieves state-of-the-art performance across all four mixed-memory settings, consistently outperforming both EverMemOS and mem0. Specifically, MEMs leads EverMemOS by 4.2 pp on LoCoMo0&OS (54.6 vs. 50.4), 1.1 pp on LoCoMo1&ALFWorld (62.4 vs. 61.3), 1.7 pp on LoCoMo2&KG (57.3 vs. 55.6), and 1.4 pp on LoCoMo3&DB (62.6 vs. 61.2). The gap to mem0 is larger: 14.5, 3.3, 8.0, and 6.0 pp respectively. The consistent gains confirm that coordinating episodic and procedural memory stores is more effective than any single-memory approach when task boundaries are blurred.

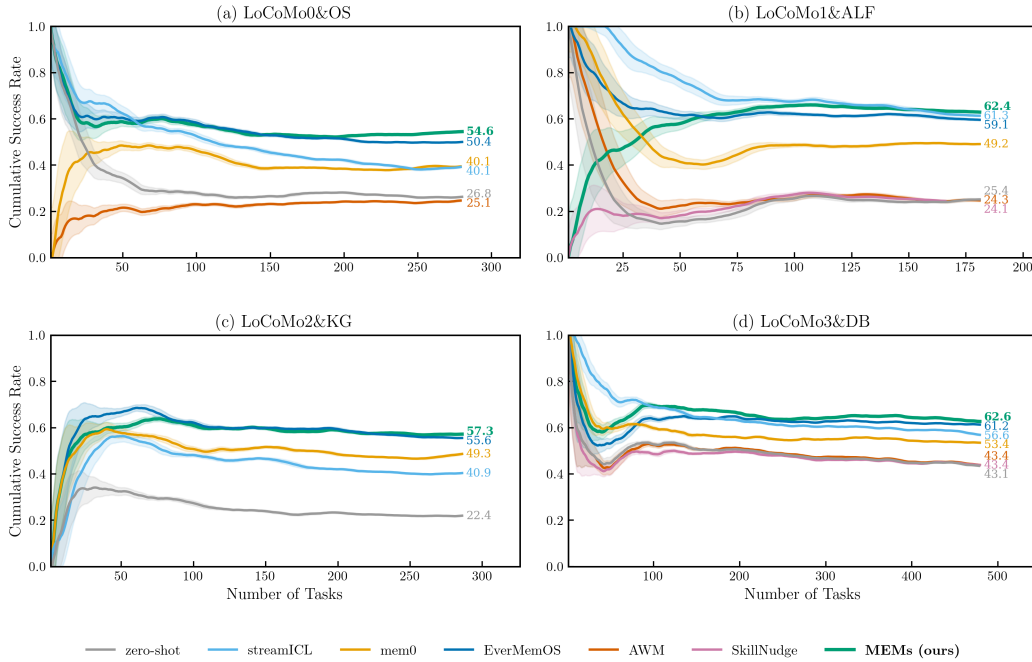


Figure 3: Cumulative Success Rate (CSR) on mixed-memory settings, where episodic tasks (LoCoMo0-3) and procedural tasks (OS, ALFWorld, KG, and DB) are interleaved in a single stream. **MEMs (ours)** achieves the highest final CSR across all completed settings.

5 Conclusion

We presented **AgentMemoryBench**, the first benchmark to jointly evaluate procedural and episodic memory under a unified continual-learning framework, spanning six datasets, four environment types, and five complementary evaluation modes. A central finding is that standard offline/online metrics are insufficient to characterize real memory behavior: StreamICL, despite matching dedicated memory methods on ASR, exhibits the highest forgetting rate, the most negative forward transfer, and the only negative repair gain among all baselines—underscoring the necessity of continual-learning diagnostics in stream-based evaluation. In mixed-memory settings, even the strongest single-memory baselines degrade substantially; our proposed **MEMs** outperforms the best single-memory baseline (EverMemOS) by 1.1–4.2 pp and surpasses mem0 and StreamICL by an average of ~ 15 pp, demonstrating that coordinating specialized memory stores and more general memory methods are essential when task boundaries are blurred. We hope **AgentMemoryBench** serves as a stepping stone toward more realistic evaluation of agent memory, and that its five-mode protocol and mixed-memory setting inspire both future memory method design and benchmark development.

272 **Limitations.** Results are limited to a single backbone (Qwen3.5-27B) with a fixed random seed;
 273 and WebShop and cross-environment transfer (TG) are pending due to compute constraints.

274 References

- 275 Qingyao Ai, Yichen Tang, Changyue Wang, Jianming Long, Weihang Su, and Yiqun Liu. Mem-
 276 orybench: A benchmark for memory and continual learning in llm systems, 2025. URL <https://arxiv.org/abs/2510.17281>.
 277
- 278 Ding Chen, Simin Niu, Kehang Li, Peng Liu, Xiangping Zheng, Bo Tang, Xinchu Li, Feiyu Xiong,
 279 and Zhiyu Li. Halumem: Evaluating hallucinations in memory systems of agents, 2026. URL
 280 <https://arxiv.org/abs/2511.03506>.
- 281 Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building
 282 production-ready ai agents with scalable long-term memory, 2025. URL <https://arxiv.org/abs/2504.19413>.
 283
- 284 Shihan Dou, Ming Zhang, Zhangyue Yin, Chenhao Huang, Yujiong Shen, Junzhe Wang, Jiayi Chen,
 285 Yuchen Ni, Junjie Ye, Cheng Zhang, Huaibing Xie, Jianglu Hu, Shaolei Wang, Weichao Wang,
 286 Yanling Xiao, Yiting Liu, Zenan Xu, Zhen Guo, Pluto Zhou, Tao Gui, Zuxuan Wu, Xipeng Qiu,
 287 Qi Zhang, Xuanjing Huang, Yu-Gang Jiang, Di Wang, and Shunyu Yao. Cl-bench: A benchmark
 288 for context learning, 2026. URL <https://arxiv.org/abs/2602.03587>.
- 289 Saman Forouzandeh, Wei Peng, Parham Moradi, Xinghuo Yu, and Mahdi Jalili. Learning hierarchical
 290 procedural memory for llm agents through bayesian selection and contrastive refinement, 2025.
 291 URL <https://arxiv.org/abs/2512.18950>.
- 292 Chuanrui Hu, Xingze Gao, Zuyi Zhou, Dannong Xu, Yi Bai, Xintong Li, Hui Zhang, Tong Li, Chong
 293 Zhang, Lidong Bing, and Yafeng Deng. Evermemos: A self-organizing memory operating system
 294 for structured long-horizon reasoning, 2026a. URL <https://arxiv.org/abs/2601.02163>.
- 295 Yuanzhe Hu, Yu Wang, and Julian McAuley. Evaluating memory in llm agents via incremental
 296 multi-turn interactions, 2026b. URL <https://arxiv.org/abs/2507.05257>.
- 297 Yuyang Hu, Shichun Liu, Yanwei Yue, Guibin Zhang, Boyang Liu, Fangyi Zhu, Jiahang Lin, Honglin
 298 Guo, Shihan Dou, Zhiheng Xi, Senjie Jin, Jiejun Tan, Yanbin Yin, Jiongnan Liu, Zeyu Zhang,
 299 Zhongxiang Sun, Yutao Zhu, Hao Sun, Boci Peng, Zhenrong Cheng, Xuanbo Fan, Jiaxin Guo,
 300 Xinlei Yu, Zhenhong Zhou, Zewen Hu, Jiahao Huo, Junhao Wang, Yuwei Niu, Yu Wang, Zhenfei
 301 Yin, Xiaobin Hu, Yue Liao, Qiankun Li, Kun Wang, Wangchunshu Zhou, Yixin Liu, Dawei
 302 Cheng, Qi Zhang, Tao Gui, Shirui Pan, Yan Zhang, Philip Torr, Zhicheng Dou, Ji-Rong Wen,
 303 Xuanjing Huang, Yu-Gang Jiang, and Shuicheng Yan. Memory in the age of ai agents, 2026c.
 304 URL <https://arxiv.org/abs/2512.13564>.
- 305 Bowen Jiang, Yuan Yuan, Maohao Shen, Zhuoqun Hao, Zhangchen Xu, Zichen Chen, Ziyi Liu,
 306 Anvesh Rao Vijjini, Jiashu He, Hanchao Yu, Radha Poovendran, Gregory Wornell, Lyle Ungar, Dan
 307 Roth, Sihao Chen, and Camillo Jose Taylor. Personamem-v2: Towards personalized intelligence
 308 via learning implicit user personas and agentic memory, 2025. URL <https://arxiv.org/abs/2512.06688>.
 309
- 310 Hanqi Jiang, Junhao Chen, Yi Pan, Ling Chen, Weihang You, Yifan Zhou, Ruidong Zhang, Lin Zhao,
 311 Yohannes Abate, and Tianming Liu. Synapse: Empowering llm agents with episodic-semantic
 312 memory via spreading activation, 2026. URL <https://arxiv.org/abs/2601.02744>.
- 313 Cheng Jiayang, Dongyu Ru, Lin Qiu, Yiyang Li, Xuezhi Cao, Yangqiu Song, and Xunliang Cai.
 314 Amemgym: Interactive memory benchmarking for assistants in long-horizon conversations, 2026.
 315 URL <https://arxiv.org/abs/2603.01966>.
- 316 Han Li, Letian Zhu, Bohan Zhang, Rili Feng, Jiaming Wang, Yue Pan, Earl T. Barr, Federica Sarro,
 317 Zhaoyang Chu, and He Ye. Contextbench: A benchmark for context retrieval in coding agents,
 318 2026. URL <https://arxiv.org/abs/2602.05892>.

319 Zhiyu Li, Chenyang Xi, Chunyu Li, Ding Chen, Boyu Chen, Shichao Song, Simin Niu, Hanyu
 320 Wang, Jiawei Yang, Chen Tang, Qingchen Yu, Jihao Zhao, Yezhaohui Wang, Peng Liu, Zehao
 321 Lin, Pengyuan Wang, Jiahao Huo, Tianyi Chen, Kai Chen, Kehang Li, Zhen Tao, Huayi Lai,
 322 Hao Wu, Bo Tang, Zhengren Wang, Zhaoxin Fan, Ningyu Zhang, Linfeng Zhang, Junchi Yan,
 323 Mingchuan Yang, Tong Xu, Wei Xu, Huajun Chen, Haofen Wang, Hongkang Yang, Wentao Zhang,
 324 Zhi-Qin John Xu, Siheng Chen, and Feiyu Xiong. Memos: A memory os for ai system, 2025.
 325 URL <https://arxiv.org/abs/2507.03724>.

326 Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding,
 327 Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui
 328 Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang.
 329 Agentbench: Evaluating llms as agents, 2025. URL <https://arxiv.org/abs/2308.03688>.

330 Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei
 331 Fang. Evaluating very long-term conversational memory of llm agents, 2024. URL <https://arxiv.org/abs/2402.17753>.

332 mem-eval-suite. LoCoMo refined. https://github.com/mem-eval-suite/LoCoMo_refined,
 333 2026. Accessed: 2026.

335 Siru Ouyang, Jun Yan, I-Hung Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Rujun Han, Long T. Le,
 336 Samira Daruki, Xiangru Tang, Vishy Tirumalashetty, George Lee, Mahsan Rofouei, Hangfei Lin,
 337 Jiawei Han, Chen-Yu Lee, and Tomas Pfister. Reasoningbank: Scaling agent self-evolving with
 338 reasoning memory, 2025. URL <https://arxiv.org/abs/2509.25140>.

339 Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E.
 340 Gonzalez. Memgpt: Towards llms as operating systems, 2024. URL <https://arxiv.org/abs/2310.08560>.

342 Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. Zep: A
 343 temporal knowledge graph architecture for agent memory, 2025. URL <https://arxiv.org/abs/2501.13956>.

345 Nous Research. Hermes agent: The agent that grows with you. [https://github.com/](https://github.com/NousResearch/hermes-agent)
 346 [NousResearch/hermes-agent](https://github.com/NousResearch/hermes-agent), 2026. Accessed: 2026-05-07.

347 Yiting Shen, Kun Li, Wei Zhou, and Songlin Hu. Mem2actbench: A benchmark for evaluating
 348 long-term memory utilization in task-oriented autonomous agents, 2026. URL <https://arxiv.org/abs/2601.19935>.

350 Haizhou Shi, Zihao Xu, Hengyi Wang, Weiyi Qin, Wenyan Wang, Yibin Wang, Zifeng Wang, Sayna
 351 Ebrahimi, and Hao Wang. Continual learning of large language models: A comprehensive survey,
 352 2024. URL <https://arxiv.org/abs/2404.16789>.

353 Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Côté, Yonatan Bisk, Adam Trischler, and Matthew
 354 Hausknecht. Alfworld: Aligning text and embodied environments for interactive learning, 2021.
 355 URL <https://arxiv.org/abs/2010.03768>.

356 Lukas Thede, Karsten Roth, Matthias Bethge, Zeynep Akata, and Tom Hartvigsen. Wikibigedit:
 357 Understanding the limits of lifelong knowledge editing in llms, 2025. URL <https://arxiv.org/abs/2503.05683>.

359 Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. Agent workflow memory, 2024.
 360 URL <https://arxiv.org/abs/2409.07429>.

361 Tianxin Wei, Noveen Sachdeva, Benjamin Coleman, Zhankui He, Yuanchen Bei, Xuying Ning,
 362 Mengting Ai, Yunzhe Li, Jingrui He, Ed H. Chi, Chi Wang, Shuo Chen, Fernando Pereira, Wang-
 363 Cheng Kang, and Derek Zhiyuan Cheng. Evo-memory: Benchmarking llm agent test-time learning
 364 with self-evolving memory, 2025. URL <https://arxiv.org/abs/2511.20857>.

365 Cheng-Kuang Wu, Zhi Rui Tam, Chieh-Yen Lin, Yun-Nung Chen, and Hung yi Lee. Streambench:
 366 Towards benchmarking continuous improvement of language agents, 2024. URL <https://arxiv.org/abs/2406.08747>.

368 Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. Longmemeval:
369 Benchmarking chat assistants on long-term interactive memory, 2025a. URL <https://arxiv.org/abs/2410.10813>.
370

371 Tingyu Wu, Zhisheng Chen, Ziyang Weng, Shuhe Wang, Chenglong Li, Shuo Zhang, Sen Hu, Silin
372 Wu, Qizhen Lan, Huacan Wang, and Ronghao Chen. Knowme-bench: Benchmarking person
373 understanding for lifelong digital companions, 2026. URL <https://arxiv.org/abs/2601.04745>.
374

375 Yaxiong Wu, Sheng Liang, Chen Zhang, Yichao Wang, Yongyue Zhang, Huifeng Guo, Ruiming
376 Tang, and Yong Liu. From human memory to ai memory: A survey on memory mechanisms in the
377 era of llms, 2025b. URL <https://arxiv.org/abs/2504.15965>.

378 Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. Webshop: Towards scalable
379 real-world web interaction with grounded language agents, 2023. URL <https://arxiv.org/abs/2207.01206>.
380

381 Yi Yu, Liuyi Yao, Yuexiang Xie, Qingquan Tan, Jiaqi Feng, Yaliang Li, and Libing Wu. Agentic
382 memory: Learning unified long-term and short-term memory management for large language
383 model agents, 2026. URL <https://arxiv.org/abs/2601.01885>.

384 Guibin Zhang, Muxin Fu, and Shuicheng Yan. Memgen: Weaving generative latent memory for
385 self-evolving agents, 2025. URL <https://arxiv.org/abs/2509.24704>.

386 Ningyu Zhang, Yunzhi Yao, Bozhong Tian, Peng Wang, Shumin Deng, Mengru Wang, Zekun Xi,
387 Shengyu Mao, Jintian Zhang, Yuansheng Ni, Siyuan Cheng, Ziwen Xu, Xin Xu, Jia-Chen Gu,
388 Yong Jiang, Pengjun Xie, Fei Huang, Lei Liang, Zhiqiang Zhang, Xiaowei Zhu, Jun Zhou, and
389 Huajun Chen. A comprehensive study of knowledge editing for large language models, 2024. URL
390 <https://arxiv.org/abs/2401.01286>.

391 Shengtao Zhang, Jiaqian Wang, Ruiwen Zhou, Junwei Liao, Yuchen Feng, Weinan Zhang, Ying Wen,
392 Zhiyu Li, Feiyu Xiong, Yutao Qi, Bo Tang, and Muning Wen. Memrl: Self-evolving agents via
393 runtime reinforcement learning on episodic memory, 2026a. URL <https://arxiv.org/abs/2601.03192>.
394

395 Zhe Zhang, Runlin Liu, Aishan Liu, Xingyu Liu, Xiang Gao, and Hailong Sun. Code2bench: Scaling
396 source and rigor for dynamic benchmark construction, 2026b. URL <https://arxiv.org/abs/2508.07180>.
397

398 Junhao Zheng, Xidi Cai, Qiuke Li, Duzhen Zhang, ZhongZhi Li, Yingying Zhang, Le Song, and
399 Qianli Ma. Lifelongagentbench: Evaluating llm agents as lifelong learners, 2025. URL <https://arxiv.org/abs/2505.11942>.
400

401 Xiaoquan Zhi, Hongke Zhao, Likang Wu, Chuang Zhao, and Hengshu Zhu. Reinventing clinical
402 dialogue: Agentic paradigms for llm enabled healthcare communication, 2025. URL <https://arxiv.org/abs/2512.01453>.
403

A Memory Benchmarks for Agents: Extended Survey

Related work on benchmarks for agent memory focuses on two primary dimensions: the evaluation paradigm and the categorization of memory types.

Streaming evaluation, introduced by StreamBench [Wu et al., 2024], is a paradigm where data and tasks are presented incrementally. In this setting, the model must complete learning and assessment immediately upon receiving each batch of the stream, rather than undergoing centralized training followed by a unified test. This approach mirrors the sequential task evaluation commonly used in continual learning (CL) [Shi et al., 2024]. The significance of streaming evaluation lies in its ability to quantify performance gains over time, memory retention, forgetting effects, generalization, and knowledge conflict resolution. However, a substantial portion of existing memory benchmarks still lacks support for streaming evaluation. While some benchmarks, such as LifelongAgentBench [Zheng et al., 2025] and Evo-Memory [Wei et al., 2025], have begun implementing stream-based assessments for procedural memory, they lack a comprehensive suite of continual learning metrics, such as backward transfer (forgetting rate) and forward transfer [Shi et al., 2024], as well as indicators for knowledge conflict resolution, such as the correction success rate used in knowledge editing [Thede et al., 2025].

Agent memory is further categorized into episodic memory and procedural memory [Wu et al., 2025b]. The former focuses on personalized user information, while the latter retains data related to task execution. Procedural memory benchmarks evaluate an agent’s ability to extract experience from past cases and internalize information over the long term, with the goal of increasing success rates in subsequent tasks. Conversely, episodic memory benchmarks assess the ability to remember user preferences—such as dietary restrictions, schedules, and personal attributes—to enhance the interaction experience. Although some recent studies, such as MemoryBench [Ai et al., 2025], attempt to address both episodic memory (referred to as declarative or factual memory) and procedural memory (referred to as experiential memory), they rely on non-streaming evaluation methods. This creates a gap in temporal assessment and fails to provide evaluation modes capable of distinguishing persistent memory methods from in-context learning baselines that merely accumulate context within the current window.

Another recent direction is competency-oriented evaluation for memory agents. MemoryAgentBench [Hu et al., 2026b] converts long-context datasets into incremental multi-turn interactions and evaluates four memory competencies: accurate retrieval, test-time learning, long-range understanding, and selective forgetting. This framing is closely related to dynamic context learning, since the agent must incrementally absorb information and later use it. However, it is not organized around hybrid episodic–procedural memory workloads or continual-learning-style temporal probes such as replay, transfer, and repair.

A.1 Non-stream Evaluation and Episodic Memory Benchmarks

LOCOMO [Maharana et al., 2024] comprises multiple session-based and cross-session reasoning tasks designed to evaluate the long-term memory of agents within ultra-long dialogues. The dataset includes 10 extended conversations, each averaging approximately 300 turns and 9,000 tokens, requiring the agent to retain user preferences, key facts, and dialogue history across sessions with a focus on causal and temporal reasoning. Similarly, LongMemEval [Wu et al., 2025a] is tailored for multi-session user-assistant interactions, covering five core capabilities: information extraction, multi-session reasoning, temporal reasoning, knowledge updating, and abstention from unanswerable questions. DMR, introduced by MemGPT [Packer et al., 2024], focuses on the long-term memory consistency of conversational agents, specifically testing their ability to maintain and recall critical user facts across multiple chat sessions. In contrast to the precise extraction of distant information emphasized by the aforementioned benchmarks, PersonaMem-v2 [Jiang et al., 2025] focuses on the long-term memory of implicit user preferences and personalized adaptation. Utilizing 1,000 distinct user personas and over 300 daily scenarios, it constructs interaction histories up to 128k tokens. This benchmark requires agents to implicitly extract preferences from task-oriented dialogues, manage privacy protection, and handle “forgetting requests,” thereby testing implicit preference reasoning and dynamic adaptation within long contexts.

These five benchmarks primarily employ accuracy and recall metrics—such as ROUGE-L, BLEU, and NDCG@k—alongside LLM-as-judge scoring. KnowMe-Bench [Wu et al., 2026] further adopts

a hierarchical scoring system that evaluates entity accuracy at the factual level, reasoning validity at the logical level, and internal-external mapping precision at the insight level. All these frameworks are classified as episodic memory benchmarks because their evaluation objectives center on the long-term retention and adaptation of user-specific information. Whether focusing on explicit facts (LOCOMO, DMR), implicit preferences (PersonaMem-v2), interaction details (LongMemEval), or internal motivations derived from autobiographical narratives (KnowMe-Bench), these efforts aim to align agent interactions with individual user needs. Consequently, they lack assessment of procedural-task execution data. Furthermore, these benchmarks do not support the streaming evaluation of episodic memory and lack evaluation modes—such as replay and repair—that can distinguish persistent memory methods from in-context learning baselines like streamICL [Wu et al., 2024].

MemoryAgentBench [Hu et al., 2026b] is another non-stream benchmark in this line. It evaluates episodic memory through incremental memory construction and covers four competencies: accurate retrieval, test-time learning, long-range understanding, and selective forgetting. However, its evaluation remains terminal rather than streaming: the agent first builds memory from sequentially provided chunks, and performance is assessed only after the full memory has been constructed. As a result, it does not provide temporal probes such as replay, transfer, or repair for measuring retention, generalization, and post-conflict recovery over time.

A.2 Stream Evaluation and Episodic Memory Benchmarks

HaluMem [Chen et al., 2026] focuses on the operation-level hallucination assessment of memory systems, specifically targeting the full lifecycle of episodic memory, including extraction, updating, and question-answering. It detects four categories of hallucinatory behaviors—fabrication, error, conflict, and omission—when a memory system processes personalized user information. Based on the HaluMem-Medium and HaluMem-Long datasets, which contain approximately 15,000 memory points and 3,500 QA pairs spanning a 10-to-20-year dialogue timeline, the benchmark emphasizes ensuring the reliability and consistency of personalized memory through fine-grained tasks. While HaluMem utilizes standard metrics such as accuracy, recall, and LLM-as-judge scoring, it introduces additional operation-level indicators, such as memory extraction accuracy and update hallucination rates. HaluMem is classified as an episodic memory benchmark because its evaluation objectives and data are anchored in personalized user information; all memory points consist of user-specific identity traits, life events, and interpersonal relationships. Its primary goal is to guarantee the reliability of episodic memory by controlling hallucinations, thereby ensuring that agent interactions align closely with individual user needs. However, similar to other episodic memory benchmarks, HaluMem lacks evaluation modes to distinguish persistent memory methods from in-context learning baselines, and it fails to provide comprehensive continual learning metrics or knowledge conflict resolution indicators to measure the quality of memory methods across the temporal dimension.

AMemGym [Jiayang et al., 2026] is an advanced interactive episodic memory benchmark designed for on-policy evaluation of memory-driven personalization in long-horizon conversations. It addresses the limitations of static off-policy evaluation in traditional benchmarks by grounding free-form interactions in structured data: predefined user profiles (sampled from 100K personas), state evolution trajectories (capturing dynamic changes in user preferences, habits, and plans over multiple periods), and state-dependent evaluation questions. Leveraging LLM-simulated users to role-play with consistent structured states, AMemGym generates high-fidelity, scalable interaction data that reflects the true conversational consequences of an assistant’s memory choices. Classified as an episodic memory benchmark, its evaluation is centered on user-specific latent states (e.g., dietary preferences, schedule changes, life events) and their dynamic retention across extended dialogue turns. AMemGym introduces a comprehensive metric suite: overall question-answering accuracy (capturing personalization and memory integration), normalized memory scores (isolating memory performance from reasoning capabilities by normalizing against random and perfect-memory baselines), and diagnostic metrics (decomposing failures into write, read, and utilization errors to pinpoint memory bottlenecks). Additionally, it supports agent self-evolution by providing environmental feedback to refine memory management policies. Despite its innovations in interactive and dynamic evaluation, AMemGym shares similar limitations with HaluMem: it lacks evaluation modes to distinguish persistent memory methods from in-context learning baselines, and while it assesses temporal memory retention across conversation periods, it does not incorporate specialized continual learning metrics

(e.g., backward transfer, knowledge conflict correction rates) for rigorous measurement of long-term memory quality over time.

A.3 Stream Evaluation and Procedural Memory Benchmarks

StreamBench [Wu et al., 2024] is the inaugural benchmark focusing on the continuous improvement of LLM agents via streaming, evaluating their ability to learn from input-feedback sequences. It serializes static datasets into quasi-real-time data streams encompassing Text-to-SQL, Python programming, tool-use, medical diagnosis, and open-domain QA. Agents are required to receive task instances, generate outputs, and update external memory components immediately upon receiving binary feedback. However, its tasks are limited to single-turn interactions, whereas AgentMemoryBench supports multi-turn interaction sequences of up to 20 turns.

LifelongAgentBench [Zheng et al., 2025] is the first streaming benchmark specifically designed to assess the lifelong learning capabilities of LLM agents, focusing on skill reuse and transfer across three interactive environments: MySQL databases, Ubuntu operating systems, and SPARQL knowledge graphs. The benchmark constructs 22 SQL skill categories, 29 Bash command categories, and 7 KG query categories to evaluate how agents accumulate operational experience. Nevertheless, neither StreamBench nor LifelongAgentBench quantifies knowledge forgetting or backward transfer over time. Furthermore, they support only intra-environment transfer rather than cross-environment transfer or multi-task online learning—all of which are core features of AgentMemoryBench.

Evo-Memory [Wei et al., 2025] emphasizes the self-evolution of memory, evaluating the retrieval, synthesis, and dynamic updating of memory within multi-task streams. Similar to AgentMemoryBench, it supports both single-turn and multi-turn tasks (e.g., household instruction following and scientific experimentation), requiring agents to refine task-execution strategies through a “retrieve-synthesize-evolve” loop. While Evo-Memory supports multi-task online learning and cross-environment transfer, it lacks support for intra-environment transfer and replay learning modes. Crucially, it considers only a single memory type: procedural memory.

These three benchmarks are classified as procedural memory benchmarks as they focus exclusively on the retention and reuse of experience data generated during task execution to improve future success rates. They lack assessment of user preferences or personal attributes. While they utilize task-specific performance metrics (e.g., terminal Success Rate, average steps) and cumulative performance curves, they lack replay and repair evaluation modes that can distinguish persistent memory methods from in-context learning baselines. AgentMemoryBench addresses this gap by introducing a comprehensive suite of metrics, including backward transfer (forgetting rate), forward transfer [Shi et al., 2024], and knowledge conflict resolution indicators such as the correction success rate [Thede et al., 2025].

A.4 Non-stream Evaluation and Hybrid Memory Benchmarks

MemoryBench [Ai et al., 2025] is the first benchmark to explicitly cover both episodic and procedural memory evaluation, breaking the limitation of single-memory assessment. Likely the classification in [Wu et al., 2025b], MemoryBench categorizes memory into declarative and procedural types. Declarative memory corresponds to episodic memory, anchoring user-specific information such as dialogue history, preferences, and task contexts to test the agent’s ability to optimize interaction experiences. Procedural memory aligns with task-execution experience, evaluating the retention of general experience from task execution to improve future success rates. The benchmark integrates 11 cross-domain datasets spanning open-domain, legal, and academic fields, supporting four task formats: Long-in Short-out (LiSo), Short-in Long-out (SiLo), Long-in Long-out (LiLo), and Short-in Short-out (SiSo). For example, multi-turn dialogue understanding in datasets like DialSim and Locomo serves episodic memory evaluation, while legal document generation in LexEval depends on procedural memory.

Despite these innovations, MemoryBench operates under a non-streaming evaluation paradigm. Training data and feedback logs (simulated via an LLM-as-user approach) are pre-generated, and the model undergoes offline or stepwise off-policy training followed by a one-time assessment on a disjoint test set. Its evaluation metrics rely on dataset-native scores (e.g., F1, ROUGE-L) consolidated into a single 1–10 score via LLM-as-Judge, lacking temporal indicators unique to continual learning. Consequently, MemoryBench exhibits two primary deficiencies: first, the absence of streaming evaluation precludes dynamic assessment of memory retention, forgetting effects, and knowledge

transfer; second, it lacks evaluation modes such as replay and repair that can distinguish persistent memory methods from in-context learning baselines like streamICL [Wu et al., 2024].

A.5 Context-Learning Benchmarks

Context-learning benchmarks evaluate how well a model acquires and applies new knowledge from within a single session’s context window, without relying on pre-trained internal knowledge. This is fundamentally different from AgentMemoryBench’s focus: we evaluate how agents accumulate, retain, and transfer knowledge across sessions through persistent external memory. While Mem2ActBench [Shen et al., 2026], CL-bench [Dou et al., 2026], CONTEXTBENCH [Li et al., 2026], and CODE2BENCH [Zhang et al., 2026b] each advance single-session context utilization, none addresses the cross-session persistent memory evolution that real-world deployments require.

Mem2ActBench: Focused on Memory-Driven Grounding, but Limited to Retrieval. Mem2ActBench designs tasks where the model must extract “new information” (e.g., user preferences or task constraints) from fragmented, long-term dialogue chains and ground them as parameters for tool calls (e.g., budget or flight preferences). While it functions as a context-learning benchmark, its limitations are twofold: (i) *Metric Deficiency*: It relies on task completion metrics like Tool Accuracy (TA), parameter F1, and BLEU-1. It cannot distinguish whether a model has truly understood the relationship between parameters and user needs or is simply performing “pseudo-learning” via mechanical string matching; (ii) *Static Paradigm*: It does not support streaming inputs or iterative memory optimization, failing to simulate the incremental accumulation of knowledge found in real-world interactions.

CL-bench: Comprehensive Evaluation of Complex In-Context Learning. CL-bench stands as a representative benchmark in this field, focusing on how models learn from complex, uncontaminated contexts such as new domain knowledge or rule systems. It provides a robust answer to the “true learning vs. mechanical patching” problem through multi-dimensional metrics: (i) *Error Distribution*: Categorizes failures into context-ignorance, misuse, or formatting errors to locate the root of the learning failure. (ii) *Sensitivity and Effort*: Evaluates stability across context lengths (up to 65,000 tokens) and compares performance across different reasoning effort levels to verify deep comprehension. (iii) *Sequential Dependency*: With over 50% of tasks requiring results from previous steps, it forces models to demonstrate cross-step knowledge reuse.

CONTEXTBENCH: Process-Oriented Evaluation of Code Context Retrieval. As a code-specific context-learning benchmark, CONTEXTBENCH targets the “black-box” limitation of outcome-driven evaluations by focusing on the process of context retrieval in coding agents. It effectively distinguishes genuine context learning from mechanical exploration through two core designs: (i) *Gold Context Anchoring*: Expert annotators label the critical code regions (gold contexts) necessary for resolving each issue, establishing a precise benchmark for evaluating retrieval relevance; (ii) *Fine-Grained Process Metrics*: By tracking the agent’s full retrieval trajectory, it computes recall, precision, and F1 scores at three granularities (file, block, line), complemented by process dynamics metrics (efficiency, redundancy, usage drop). These metrics expose whether the agent genuinely retrieves and utilizes key context or merely achieves task success through blind trial-and-error, thus solving the “true learning vs. mechanical patching” dilemma in code-related context learning.

CODE2BENCH: Rigorous Validation of Code Generation Generalization. Focused on code-generating LLMs, CODE2BENCH addresses the core challenge of distinguishing “memorization vs. true generalization” through a “Dual Scaling” philosophy: (i) *Dynamic Contamination-Resistant Sourcing*: It extracts functions exclusively from GitHub commits post-dating the evaluated models’ knowledge cutoff, ensuring tasks are provably unseen and eliminating memorization of pre-trained data; (ii) *High-Rigor Property-Based Testing (PBT)*: Instead of relying on limited example-based tests, it generates hundreds of structured inputs (including edge cases) and enforces 100% branch coverage, exposing “near-perfect failures” (solutions passing 98% of tests but failing subtle edge cases) that conventional benchmarks miss. Additionally, it classifies tasks into Self-Contained (SC, pure algorithmic reasoning) and Weakly Self-Contained (WSC, API application), enabling targeted evaluation of distinct learning capabilities. Together, these designs ensure that the benchmark measures true code generation generalization (genuine learning) rather than mechanical pattern matching or memorization.

The Cost of Missing Hybrid Evaluation. The absence of a hybrid memory benchmark is not merely an evaluation gap—it has concrete consequences for the field. Methods that attempt to bridge procedural and episodic memory, such as MemGen [Zhang et al., 2025], which unifies heterogeneous memory representations via machine-native latent token sequences, cannot be rigorously validated: no existing benchmark supports the mixed-scenario streams in which such coordination is actually needed. Without a testbed that interleaves procedural tasks and episodic interactions with blurred boundaries, the cross-type coordination capability of general memory approaches remains unmeasured.

B Annotation Guide for Table 1

Table 1 evaluates ten benchmarks across 13 dimensions using three symbols: ✓ (fully supported), × (not supported), and □ (not applicable by design). We explain each dimension and the rationale for □ assignments below.

Memory Eval. Whether the benchmark targets *persistent external memory*—storage, retrieval, and update of knowledge across sessions or sub-tasks. ✓ denotes a memory benchmark; × denotes a context-learning benchmark, which evaluates within-session context utilization rather than persistent memory.

CL Metrics. Whether the benchmark reports continual learning metrics beyond terminal task performance, specifically Forgetting Rate (backward transfer) and/or Forward Transfer Gain. □ is assigned to benchmarks that operate under purely offline or single-session paradigms—including offline memory benchmarks such as LoCoMo, MemoryBench, and MemoryAgentBench, as well as context-learning benchmarks—where temporal dynamics are absent by design and such metrics are structurally inapplicable. **Distinguish Learn.** Whether the evaluation protocol can distinguish genuine learning from superficial retrieval or rote assembly. For memory benchmarks: ✓ denotes that the benchmark includes explicit designs or analyses aimed at verifying real learning behavior rather than simple memorization or retrieval-only behavior; × denotes no such distinguishing capability. For context-learning benchmarks: ✓ denotes the ability to distinguish genuine context learning from mechanical pattern matching within a session (e.g., CL-bench); × denotes no such distinguishing capability.

Hybrid Memory. Whether the benchmark simultaneously evaluates both procedural memory (experiential, related to task execution) and episodic memory (factual/declarative, related to user preferences). □ is assigned to context-learning benchmarks for which this categorization is not applicable.

Multi-turn. Whether individual *sub-tasks* require multiple rounds of agent–environment interaction to reach completion—for example, iterative tool invocations across steps in database or operating-system tasks. Single-turn question answering (e.g., LoCoMo sub-tasks each consist of answering one question) does not qualify, regardless of whether the overall benchmark spans many sessions.

Tool-use. Whether sub-tasks require the agent to invoke external tools, such as code execution engines, SQL databases, or web APIs.

Single-task Online and Multi-task Online. Whether the benchmark supports streaming evaluation within a single task type or across interleaved task types respectively, where the agent processes sub-tasks sequentially and updates memory in real time. □ is assigned to context-learning benchmarks, for which online streaming across sessions is structurally inapplicable.

Offline. Whether the benchmark supports static train-test split evaluation, where memory is built from a training set and assessed on a disjoint test set.

Intra-env Transfer, Cross-env Transfer, Replay, and Repair correspond directly to AgentMemory-Bench’s four advanced evaluation modes (Section ??). □ is assigned to benchmarks for which these continual-memory evaluation modes are not applicable by design, such as context-learning benchmarks. × is assigned to memory benchmarks that do not implement these specific modes, including both non-stream memory benchmarks and online memory benchmarks lacking these evaluations.

C Memory Methodologies for Agents: Extended Survey

Memory mechanisms can be categorized into two primary classes based on their functional focus: procedural memory methods and episodic memory methods.

Procedural memory methods focus on extracting reusable knowledge from task execution experiences. SYNAPSE [Jiang et al., 2026] integrates episodic memory from interactions with abstract semantic concepts by constructing a unified episodic-semantic graph and a spreading activation mechanism, which dynamically highlights relevant knowledge subgraphs to enhance reliability in complex temporal and multi-hop reasoning tasks. AgeMem [Yu et al., 2026] utilizes a unified tool-like interface and a three-stage progressive reinforcement learning strategy, enabling large language models (LLMs) to autonomously perform operations such as storage, retrieval, and updating of long-term and short-term memory. This approach learns collaborative management strategies from multi-stage task experiences to improve adaptation and context utilization in long-term reasoning. MACLA [Forouzandeh et al., 2025] builds an external hierarchical program memory that tracks program reliability through Bayesian selection and refines preconditions and action sequences via contrastive learning. It extracts interpretable reusable programs from massive trajectories to improve sample efficiency and generalization in long-horizon tasks. Agent Workflow Memory (AWM) [Wang et al., 2024] records and retrieves complete workflow histories to extract standardized execution paths and key nodes from repetitive tasks, reducing redundant decision steps and improving consistency by solidifying procedural experiences into reusable templates. ReasoningBank [Ouyang et al., 2025] focuses on distilling generalizable high-level reasoning strategies from both successful and failed task experiences to construct a specialized reasoning memory framework. MemRL [Zhang et al., 2026a] applies non-parametric reinforcement learning to episodic memory to achieve self-evolution based on runtime environmental feedback. By decoupling the stable reasoning of a frozen LLM from the plasticity of external memory, it distinguishes high-value strategies from semantic noise. It constructs a structured memory bank of intent-experience-utility (Q-value) triplets and employs a two-stage retrieval mechanism involving semantic similarity recall and value-aware selection, while updating memory utility via Bellman updates during runtime.

These methods share the common characteristic of centering on task execution experience, transforming it into reusable knowledge through structural modeling, abstraction, and intelligent retrieval. However, they generally assume clear task boundaries and explicit task types. In hybrid scenarios where episodic and procedural memory tasks are intertwined, these methods struggle with flexible memory scheduling and lack the dynamic adaptability required for fuzzy task boundaries.

Episodic memory methods prioritize user preference retention and context understanding during long-term dialogues. Mem0 [Chhikara et al., 2025] provides a scalable long-term memory system that dynamically captures and integrates key information through an extraction-update pipeline, supporting graph-structured storage to model complex semantic relationships and maintain consistency in cross-session reasoning. MemGPT [Packer et al., 2024] adopts an operating-system-inspired hierarchical memory architecture, partitioning memory into a main context and an external context. It implements information paging via function calls to efficiently manage user facts and dialogue history, supporting long-document analysis and multi-session interactions. ZEP [Rasmussen et al., 2025] builds a memory layer service based on temporal knowledge graphs, using a hierarchical structure of episodic, semantic entity, and community subgraphs to dynamically integrate unstructured conversation data with structured business data. MemOS [Li et al., 2025] introduces a memory operating system to manage heterogeneous memory types, including parametric, active, and plaintext memory, using MemCube as the core unit for lifecycle scheduling and cross-platform migration. EverMemOS [Hu et al., 2026a] constructs a self-organizing memory system that mimics the biological memory cycle through stages of MemCell generation, semantic integration into MemScenes, and reconstructive recall. While effective in their respective domains, these five methods are restricted to the episodic memory dimension and cannot support the transfer of task execution experiences. Evaluation of these methods typically relies on episodic memory benchmarks such as Locomo [Maharana et al., 2024], LongMemEval [Wu et al., 2025a], and DMR [Packer et al., 2024], which focus on user preferences and temporal reasoning but fail to assess performance in procedural task execution or hybrid scenarios.

Taken together, procedural memory methods excel within task-execution domains but assume clear task boundaries and cannot adapt to user-specific context; episodic memory methods model user preferences effectively but cannot generalize procedural experience across tasks. Neither type alone

is sufficient when procedural tasks and episodic interactions are interleaved—the precise condition that real-world deployments impose. MEMs addresses this by retaining both specialized stores and introducing a lightweight trigger model as a coordination layer, enabling each store to remain structurally optimized for its domain while the router decides when and where to retrieve and update.

D Memory lifecycle formulation

Each **task** in AgentMemoryBench corresponds to a **dataset**; each dataset contains multiple **sub-tasks**—individual samples requiring single-turn or multi-turn agent–environment interactions.

To systematically evaluate memory systems across these datasets and evaluation modes, we establish a formal framework defining the agent–environment interaction protocol. This formulation specifies how memory is retrieved before each sub-task and how new experiences are transformed into persistent memory upon completion.

D.1 Single sub-task trajectory definition

We formalize memory evaluation in large language model (LLM) based agents as follows. Consider an agent executing a sub-task within an environmental state space S . The state evolves according to a stochastic transition model Ψ :

$$s_{t+1} \sim \Psi(s_{t+1}|s_t, a_t)$$

where a_t denotes the action at each time step t . At each step t , the agent receives an observation:

$$o_t = O(h_t, Q)$$

Here, h_t represents the interaction history visible during the current sub-task, typically the agent–environment trajectory. Q denotes sub-task requirements, including the system prompt, toolset T , and objective. O is the observation function that constructs the agent’s current view by combining interaction history with task requirements. The agent selects actions following a policy:

$$a_t = \pi(o_t, m_t), a_t \in T$$

where m_t is the memory signal retrieved from system M_t :

$$m_t = R(M_t, o_t)$$

The retrieval operator R identifies relevant content based on the observation and returns a formatted signal, such as text snippets or summaries, for the LLM policy.

D.2 Memory system lifecycle

The memory lifecycle during sub-task k execution is examined below. The memory system $M_t \in \mathcal{M}$, where $\mathcal{M}$ is the space of permissible configurations, evolves through three operators. The memory retrieval operator R extracts relevant information at time t based on the observation and memory state M_t :

$$m_t = R(M_t, o_t), \quad t \geq 0$$

The memory formation operator F transforms informational artifacts ϕ_t generated after an action into memory candidates. These artifacts may include tool outputs, reasoning, plans, self-evaluations, or environmental feedback:

$$M_{t+1}^{form} = F(M_t, \phi_t), \quad t \geq 0$$

The memory evolution operator E integrates candidates into the repository. The evolved state M_{t+1} persists for subsequent decisions and sub-tasks:

$$M_{t+1} = E(M_{t+1}^{form}, M_t), \quad t \geq 0$$

Most studies (e.g., StreamBench [Wu et al., 2024], LifelongAgentBench [Zheng et al., 2025], EvoMemory [Wei et al., 2025]) adopt a sub-task-level update hypothesis. Instead of step-wise invocation, R is called once before sub-task k . Upon completion, F and E are executed sequentially:

$$m_k = R(M_k, Q_k), \quad M_{k+1}^{form} = F(M_k, \phi_k), \quad M_{k+1} = E(M_{k+1}^{form}, M_k)$$

where M_k is the state at the onset of sub-task k , Q_k denotes requirements, and M_{k+1} is the state after completion. Term ϕ_k refers to artifacts produced at the conclusion, such as outcomes, feedback, or interaction history.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [\[Yes\]](#)

Justification: The abstract and introduction clearly state the three contributions: the Agent-MemoryBench benchmark, the MEMs methodology, and the experimental findings on memory contamination and collapse in mixed-memory settings. All claims are supported by results in Section 4.

Guidelines:

- The answer [\[N/A\]](#) means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [\[No\]](#) or [\[N/A\]](#) answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [\[Yes\]](#)

Justification: A dedicated “Limitations” paragraph is included at the end of Section 5, discussing the single-backbone/single-seed setup, incomplete results for WebShop and cross-environment transfer due to compute budget, and the structural ceiling of LoCoMo dialogues fitting within modern context windows.

Guidelines:

- The answer [\[N/A\]](#) means that the paper has no limitation while the answer [\[No\]](#) means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: This paper does not include theoretical results or proofs; all contributions are empirical (benchmark design, methodology, and experiments).

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Section 4.1 specifies all baselines, hyperparameters, data splits, model backbones, and random seeds. Code is released at the anonymous repository linked in the abstract.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).

- (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: Code is released anonymously at the link provided in the abstract. All six datasets (DB, OS, KG, ALFWorld, WebShop, LoCoMo) are publicly available through their original papers.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: Section 4.1 reports all hyperparameters for each memory method, data splits, evaluation mode configurations, model backbones, and a fixed random seed (66) for all experiments.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [No]

Justification: Error bars are not reported due to the high computational cost of running multiple seeds across six datasets and five evaluation modes. A fixed random seed (66) is used throughout and reported in all table captions.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [No]

Justification: Detailed compute resource information (GPU type, memory, runtime) is not included in the paper. The backbone models used (Qwen3.5-27B, GPT-4o-mini, Qwen3-14B) are specified in each table caption.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: The research conforms with the NeurIPS Code of Ethics. No human subjects, harmful data collection, or dual-use concerns are involved.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [N/A]

Justification: This is a benchmark and evaluation methodology paper. It does not introduce deployable models or systems with direct societal risk; no negative societal impacts are anticipated beyond those of the underlying LLM-based agents it evaluates.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The paper does not release pre-trained models or scraped datasets with high misuse risk. Only benchmark evaluation code and pointers to existing publicly available datasets are released.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

1005 Justification: All datasets and baseline methods are cited with their original papers and
1006 used under their respective licenses: ALFWorld (MIT), WebShop (MIT), SkillNudge/Her-
1007 mesAgent (MIT), AgentBench/DB/OS/KG (Apache 2.0), mem0 (Apache 2.0), AWM
1008 (Apache 2.0), EverMemOS (Apache 2.0), StreamICL/StreamBench (Apache 2.0), and
1009 LoCoMo/LoCoMo-refined (CC BY-NC 4.0).

1010 Guidelines:

- 1011 • The answer [N/A] means that the paper does not use existing assets.
- 1012 • The authors should cite the original paper that produced the code package or dataset.
- 1013 • The authors should state which version of the asset is used and, if possible, include a
1014 URL.
- 1015 • The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- 1016 • For scraped data from a particular source (e.g., website), the copyright and terms of
1017 service of that source should be provided.
- 1018 • If assets are released, the license, copyright information, and terms of use in the
1019 package should be provided. For popular datasets, `paperswithcode.com/datasets`
1020 has curated licenses for some datasets. Their licensing guide can help determine the
1021 license of a dataset.
- 1022 • For existing datasets that are re-packaged, both the original license and the license of
1023 the derived asset (if it has changed) should be provided.
- 1024 • If this information is not available online, the authors are encouraged to reach out to
1025 the asset’s creators.

1026 13. New assets

1027 Question: Are new assets introduced in the paper well documented and is the documentation
1028 provided alongside the assets?

1029 Answer: [Yes]

1030 Justification: AgentMemoryBench code is released at the anonymous repository linked in
1031 the abstract, with usage instructions for reproducing all experiments.

1032 Guidelines:

- 1033 • The answer [N/A] means that the paper does not release new assets.
- 1034 • Researchers should communicate the details of the dataset/code/model as part of their
1035 submissions via structured templates. This includes details about training, license,
1036 limitations, etc.
- 1037 • The paper should discuss whether and how consent was obtained from people whose
1038 asset is used.
- 1039 • At submission time, remember to anonymize your assets (if applicable). You can either
1040 create an anonymized URL or include an anonymized zip file.

1041 14. Crowdsourcing and research with human subjects

1042 Question: For crowdsourcing experiments and research with human subjects, does the paper
1043 include the full text of instructions given to participants and screenshots, if applicable, as
1044 well as details about compensation (if any)?

1045 Answer: [N/A]

1046 Justification: The paper does not involve crowdsourcing or research with human subjects.

1047 Guidelines:

- 1048 • The answer [N/A] means that the paper does not involve crowdsourcing nor research
1049 with human subjects.
- 1050 • Including this information in the supplemental material is fine, but if the main contribu-
1051 tion of the paper involves human subjects, then as much detail as possible should be
1052 included in the main paper.
- 1053 • According to the NeurIPS Code of Ethics, workers involved in data collection, curation,
1054 or other labor should be paid at least the minimum wage in the country of the data
1055 collector.

1056	15. Institutional review board (IRB) approvals or equivalent for research with human subjects
1057	
1058	Question: Does the paper describe potential risks incurred by study participants, whether
1059	such risks were disclosed to the subjects, and whether Institutional Review Board (IRB)
1060	approvals (or an equivalent approval/review based on the requirements of your country or
1061	institution) were obtained?
1062	Answer: [N/A]
1063	Justification: The paper does not involve human subjects research.
1064	Guidelines:
1065	• The answer [N/A] means that the paper does not involve crowdsourcing nor research
1066	with human subjects.
1067	• Depending on the country in which research is conducted, IRB approval (or equivalent)
1068	may be required for any human subjects research. If you obtained IRB approval, you
1069	should clearly state this in the paper.
1070	• We recognize that the procedures for this may vary significantly between institutions
1071	and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the
1072	guidelines for their institution.
1073	• For initial submissions, do not include any information that would break anonymity (if
1074	applicable), such as the institution conducting the review.
1075	16. Declaration of LLM usage
1076	Question: Does the paper describe the usage of LLMs if it is an important, original, or
1077	non-standard component of the core methods in this research? Note that if the LLM is used
1078	only for writing, editing, or formatting purposes and does <i>not</i> impact the core methodology,
1079	scientific rigor, or originality of the research, declaration is not required.
1080	Answer: [Yes]
1081	Justification: LLMs are core components of the benchmark evaluation pipeline: Qwen3.5-
1082	27B serves as the main agent backbone, GPT-4o-mini as the auxiliary memory model, and
1083	Qwen3-14B as the judge for LoCoMo tasks. All are described in Section 4.1 and individual
1084	table captions.
1085	Guidelines:
1086	• The answer [N/A] means that the core method development in this research does not
1087	involve LLMs as any important, original, or non-standard components.
1088	• Please refer to our LLM policy in the NeurIPS handbook for what should or should not
1089	be described.